

LYKE Mobile Application - Implementation Plan

Technology Stack

Layer	Technology
Backend API	C# ASP.NET Core 10.0
ORM	Entity Framework Core 9.0
Database	PostgreSQL 16
Frontend	Ionic 7 + Angular 17
Authentication	ASP.NET Core Identity + JWT
File Storage	Azure Blob Storage / AWS S3
Search	PostgreSQL Full-Text Search (MVP)
Caching	Redis
Analytics	Application Insights / Custom Events

Current Progress Summary

Last Updated: 2026-02-04

Component	Status
Backend Foundation	☑ Complete
Database Schema & Migrations	☑ Complete
Entity Configurations	☑ Complete (15 entities)
Repository Pattern	☑ Complete
Exception Handling	☑ Complete
JWT Infrastructure	☑ Complete
Auth Service & Endpoints	☑ Complete (7 endpoints)
Profile Service & Endpoints	☑ Complete (8 endpoints)
Feed Service & Endpoints	☑ Complete (8 endpoints)
Commerce Service & Endpoints	☑ Complete (6 endpoints)
FluentValidation	☑ Complete (11 validators)
Seed Data (Lookups)	☑ Complete (8 body types, 12 fit tags)
Frontend	✗ Not Started

Component	Status
Tests	✗ Not Started
Overall Backend Progress: ~60% Overall Project: ~30%	

Phase 1: Project Foundation & Infrastructure

1.1 Backend Project Setup

- ☒ Create ASP.NET Core Web API solution structure
 - `Lyke.Api` - Web API project
 - `Lyke.Core` - Domain entities and interfaces
 - `Lyke.Infrastructure` - EF Core, repositories, external services
 - `Lyke.Application` - Business logic, DTOs, services
- ☒ Configure PostgreSQL connection and EF Core
- ☒ Set up dependency injection container
- ☒ Configure logging (Serilog)
- ☒ Set up environment-based configuration (appsettings.json)
- ☒ Add health check endpoints
- ☒ Configure CORS for mobile app

1.2 Frontend Project Setup

- ☐ Initialize Ionic Angular project (`ionic start lyke-app blank --type=angular`)
- ☐ Configure project structure
 - `/src/app/core` - Services, guards, interceptors
 - `/src/app/shared` - Shared components, pipes, directives
 - `/src/app/features` - Feature modules (feed, profile, creator, etc.)
 - `/src/app/models` - TypeScript interfaces/models
- ☐ Set up environment configuration (dev/staging/prod)
- ☐ Configure HTTP interceptors for auth tokens
- ☐ Set up state management (NgRx or simple services)
- ☐ Configure Capacitor for native builds

1.3 CI/CD Pipeline

- ☐ Set up GitHub Actions / Azure DevOps pipeline
- ☐ Configure build stages (build, test, deploy)
- ☐ Set up database migrations automation
- ☐ Configure environment deployments

Phase 2: Database Design & Entity Framework Models

2.1 Core Entities

Users

- Id (GUID)
- Email
- PasswordHash
- UserType (Shopper, Creator, Retailer, Admin)
- CreatedAt
- UpdatedAt
- IsActive

BodyProfiles

- Id (GUID)
- UserId (FK)
- HeightCm (int)
- WeightKg (decimal)
- BodyTypeId (FK)
- FitPreference (enum: Fitted, Regular, Relaxed)
- CreatedAt
- UpdatedAt

BodyTypes (lookup)

- Id
- Name
- Description
- DisplayOrder

Creators

- Id (GUID)
- UserId (FK)
- DisplayName
- Bio
- IsVerified
- SocialLinks (JSON)
- PayoutDetails (encrypted)
- CreatedAt

Retailers

- Id (GUID)
- Name
- LogoUrl
- WebsiteUrl
- AffiliateConfig (JSON)
- IsActive
- CreatedAt

Products

- Id (GUID)
- RetailerId (FK)
- ExternalSku
- Name
- Description
- Category
- SubCategory
- ImageUrls (JSON array)

- ProductUrl
- Price
- Currency
- IsActive
- LastSyncedAt
- CreatedAt

Posts (Outfit Content)

- Id (GUID)
- CreatorId (FK)
- Title
- Description
- MediaType (Image, Video)
- MediaUrls (JSON array)
- Status (Draft, PendingReview, Published, Rejected)
- ModerationNotes
- PublishedAt
- CreatedAt
- UpdatedAt

PostProducts (junction)

- Id
- PostId (FK)
- ProductId (FK)
- SizeWorn
- FitNotes
- FitRating (enum: TooSmall, SlightlySmall, TrueToSize, SlightlyLarge, TooLarge)
- StylingNotes

PostFitTags (junction)

- PostProductId (FK)
- FitTagId (FK)

FitTags (lookup)

- Id
- Name (e.g., "Tight on hips", "Long in arms")
- Category
- IsActive

Engagements

- Id (GUID)
- UserId (FK)
- PostId (FK)
- Type (View, Like, Save, Share)
- CreatedAt

ClickEvents

- Id (GUID)
- UserId (FK, nullable)
- PostId (FK)
- PostProductId (FK)
- SessionId
- CreatedAt
- ConvertedAt (nullable)

```
└─ AttributionData (JSON)

SponsoredPlacements
├─ Id (GUID)
├─ RetailerId (FK)
├─ ProductId (FK, nullable)
├─ BudgetAmount
├─ SpentAmount
├─ TargetBodyTypes (JSON array)
├─ TargetCategories (JSON array)
├─ StartDate
├─ EndDate
├─ IsActive
├─ CreatedAt

CreatorEarnings
├─ Id (GUID)
├─ CreatorId (FK)
├─ ClickEventId (FK)
├─ EarningType (Affiliate, Sponsored)
├─ Amount
├─ Currency
├─ Status (Pending, Confirmed, Paid)
├─ PaidAt
├─ CreatedAt
```

2.2 Database Tasks

- ☒ Create EF Core DbContext with all entity configurations
- ☒ Configure entity relationships and constraints
- ☒ Set up soft delete for applicable entities
- ☒ Create initial migration
- ☒ Implement audit fields (CreatedAt, UpdatedAt) via SaveChanges override
- ☒ Create database indexes for performance
 - `IX_Posts_CreatorId_Status_PublishedAt`
 - `IX_BodyProfiles_BodyTypeId_HeightCm_WeightKg`
 - `IX_Products_RetailerId_Category_IsActive`
 - `IX_ClickEvents_PostId_CreatedAt`
- ☐ Set up PostgreSQL full-text search indexes
- ☒ Create seed data for lookup tables (BodyTypes, FitTags)

Phase 3: Authentication & Authorization

3.1 Backend Auth Implementation

- ☒ Configure ASP.NET Core Identity with PostgreSQL
- ☒ Implement JWT token generation and validation
- ☒ Create refresh token mechanism with rotation
- ☒ Create AuthService with full authentication logic

- ☒ Create FluentValidation validators for all auth requests
- ☐ Implement role-based authorization (Shopper, Creator, Retailer, Admin)
- ☐ Add policy-based authorization for granular permissions
- ☐ Implement account lockout and security features

3.2 API Endpoints - Authentication (Minimal APIs)

- ☒ POST /api/auth/register - Register new user
- ☒ POST /api/auth/login - Login with email/password
- ☒ POST /api/auth/refresh - Refresh access token
- ☒ POST /api/auth/logout - Invalidate refresh token
- ☒ POST /api/auth/forgot-password - Request password reset
- ☒ POST /api/auth/reset-password - Reset password with token
- ☒ DELETE /api/auth/account - Delete account (GDPR)
- ☐ POST /api/auth/social/google - Google OAuth login
- ☐ POST /api/auth/social/apple - Apple Sign-In

3.3 Frontend Auth Implementation

- ☐ Create AuthService with token management
 - ☐ Implement HTTP interceptor for JWT injection
 - ☐ Create auth guard for protected routes
 - ☐ Build login page component
 - ☐ Build registration page component
 - ☐ Implement social login buttons (Google, Apple)
 - ☐ Create forgot/reset password flow
 - ☐ Implement secure token storage (Capacitor Secure Storage)
-

Phase 4: User Profile & Body Profile

4.1 Backend - Profile APIs (Minimal APIs)

- ☒ GET /api/profile - Get current user profile
- ☒ PUT /api/profile - Update user profile
- ☒ GET /api/profile/body - Get body profile
- ☒ POST /api/profile/body - Create body profile
- ☒ PUT /api/profile/body - Update body profile
- ☒ DELETE /api/profile/body - Delete body profile
- ☒ GET /api/lookup/body-types - Get body type options
- ☒ GET /api/lookup/fit-preferences - Get fit preference options

4.2 Backend Tasks

- ☒ Create ProfileService with business logic
- ☒ Implement body profile validation rules (FluentValidation)
- ☒ Create DTOs for profile data (never expose raw weights to other users)
- ☒ Implement profile anonymization for display (ranges/bands)
- ☒ Add profile completeness calculation

4.3 Frontend - Profile Module

- ☐ Create onboarding flow component (multi-step wizard)
 - Step 1: Basic info (email verified)
 - Step 2: Height input (with unit toggle cm/ft)
 - Step 3: Weight input (with unit toggle kg/lbs)
 - Step 4: Body type selection (visual cards)
 - Step 5: Fit preferences (optional)
 - ☐ Create profile view/edit component
 - ☐ Create body profile edit component
 - ☐ Implement profile data management (view/edit/delete)
 - ☐ Add unit conversion utilities
-

Phase 5: Content Feed & Discovery

5.1 Matching Algorithm Service

- ☒ Create BodyProfileMatchingService (integrated in FeedService)
 - Calculate similarity score between profiles
 - Weight factors: height (30%), weight (30%), body type (40%)
 - Configurable via appsettings (MatchingSettings)
- ☒ Implement feed ranking algorithm
 - Primary: Body profile similarity
 - Secondary: Recency score
 - Configurable weights and tolerances
- ☐ Create caching layer for computed matches (Redis - future)
- ☐ Implement A/B testing hooks for algorithm tuning

5.2 Backend - Feed APIs (Minimal APIs)

- ☒ GET /api/feed - Get personalized feed (paginated, filtered, sorted)
- ☒ GET /api/feed/explore - Explore/discover content (trending)
- ☒ GET /api/posts/{id} - Get post detail
- ☒ GET /api/posts/{id}/similar - Get similar posts
- ☒ POST /api/posts/{id}/engage - Record engagement (view, like, save, share)
- ☒ DELETE /api/posts/{id}/engage - Remove engagement (unlike, unsave)
- ☒ GET /api/posts/saved - Get user's saved posts
- ☒ GET /api/search - Search posts, products, creators

5.3 Backend Tasks

- ☒ Create FeedService with pagination
- ☒ Implement feed query with EF Core
- ☐ Add Redis caching for hot content
- ☒ Create filtering and sorting logic (category, retailer, fit tags)
- ☒ Implement pagination support
- ☐ Add content pre-fetching hints

5.4 Frontend - Feed Module

- ☐ Create feed page with infinite scroll
 - ☐ Build post card component
 - Creator avatar and anonymized stats
 - Media carousel (images/video)
 - Product tags overlay
 - Fit feedback summary
 - Engagement buttons
 - ☐ Create filter drawer/modal component
 - Category filter (chips)
 - Retailer filter (searchable list)
 - Fit tags filter (multi-select)
 - ☐ Create post detail page
 - Full media viewer
 - Creator profile summary (anonymized)
 - All tagged products list
 - Fit notes and feedback
 - "Shop Now" CTAs
 - ☐ Implement pull-to-refresh
 - ☐ Add skeleton loading states
 - ☐ Create saved posts page
-

Phase 6: Product Linking & Commerce

6.1 Backend - Commerce APIs

```
POST  /api/clicks/track          - Track outbound click
GET    /api/products/{id}       - Get product details
GET    /api/products/search     - Search products (for creators)
GET    /api/retailers           - List active retailers
GET    /api/retailers/{id}/products - Get retailer products
POST   /api/retailers/{id}/conversions - Conversion webhook
```

6.2 Backend Tasks

- ☒ Create CommerceService
 - Generate unique click IDs
 - Store attribution data (JSON in ClickEvent.AttributionData)
 - Handle affiliate link generation
- ☒ Implement affiliate URL builder per retailer (uses Retailer.AffiliateConfig JSON)
- ☒ Create conversion webhook endpoints (with HMAC signature verification)
- ☐ Build click analytics aggregation (deferred to Phase 10)

6.3 Frontend Tasks

- ☐ Create product card component
 - ☐ Implement click tracking before redirect
 - ☐ Build in-app browser for product views (optional)
 - ☐ Create "Shop the Look" component
 - ☐ Add deep linking support
-

Phase 7: Creator Features

7.1 Backend - Creator APIs

```
POST  /api/creators/register      - Register as creator
GET    /api/creators/profile      - Get creator profile
PUT    /api/creators/profile      - Update creator profile
POST   /api/creators/posts        - Create new post
PUT    /api/creators/posts/{id}   - Update post
DELETE /api/creators/posts/{id}   - Delete post
GET    /api/creators/posts        - Get creator's posts
POST   /api/creators/posts/{id}/submit - Submit for review
GET    /api/creators/analytics    - Get performance metrics
GET    /api/creators/earnings     - Get earnings summary
GET    /api/creators/earnings/history - Get earnings history
```

7.2 Backend Tasks

- ☐ Create CreatorService
- ☐ Implement creator verification workflow
- ☐ Build media upload service (Azure Blob/S3)
 - Image optimization and resizing
 - Video transcoding (or use external service)
 - Thumbnail generation
- ☐ Create post draft/publish workflow
- ☐ Implement product search and tagging
- ☐ Build earnings calculation service
- ☐ Create analytics aggregation queries

7.3 Frontend - Creator Module

- ☐ Create creator registration flow
- ☐ Build creator dashboard page
 - Performance overview cards
 - Recent posts list
 - Earnings summary
- ☐ Create post creation wizard
 - Step 1: Media upload (camera/gallery)
 - Step 2: Product tagging interface
 - Step 3: Size and fit details per product

- Step 4: Styling notes
 - Step 5: Preview and submit
 - ☐ Build product search and tag component
 - ☐ Create post management page (drafts, published, rejected)
 - ☐ Build analytics page with charts
 - ☐ Create earnings page with payout history
-

Phase 8: Retailer Portal (B2B)

8.1 Backend - Retailer APIs

```
POST /api/retailers/register - Register retailer
GET  /api/retailers/profile  - Get retailer profile
PUT  /api/retailers/profile  - Update retailer profile
POST /api/retailers/products/import - Import product feed
GET  /api/retailers/products - List products
PUT  /api/retailers/products/{id} - Update product
POST /api/retailers/campaigns - Create sponsored campaign
GET  /api/retailers/campaigns - List campaigns
PUT  /api/retailers/campaigns/{id} - Update campaign
GET  /api/retailers/analytics - Get engagement analytics
GET  /api/retailers/analytics/export - Export analytics CSV
GET  /api/retailers/insights/fit - Get fit feedback insights
GET  /api/retailers/insights/body-profiles - Get body profile insights
```

8.2 Backend Tasks

- ☐ Create RetailerService
- ☐ Build product feed importer
 - CSV parser
 - API endpoint for real-time updates
 - Validation and error reporting
- ☐ Create sponsored placement service
 - Budget management
 - Targeting logic
 - Impression/click tracking
- ☐ Build analytics aggregation service
 - Engagement by body profile band
 - Fit feedback trends
 - Category performance
- ☐ Implement data anonymization layer
- ☐ Create CSV/Excel export functionality

8.3 Frontend - Retailer Module (Separate Web App or Mobile)

- ☐ Create retailer dashboard
- ☐ Build product management interface

- ☐ Create campaign builder
 - ☐ Build analytics dashboard with charts
 - Engagement metrics
 - Body profile distribution
 - Fit feedback analysis
 - ☐ Create export functionality
 - ☐ Build product feed upload interface
-

Phase 9: Admin & Moderation

9.1 Backend - Admin APIs

```
GET    /api/admin/posts/pending    - Get posts pending review
POST   /api/admin/posts/{id}/approve - Approve post
POST   /api/admin/posts/{id}/reject - Reject post with reason
GET    /api/admin/users           - List users (paginated)
POST   /api/admin/users/{id}/suspend - Suspend user
POST   /api/admin/users/{id}/unsuspend - Unsuspend user
GET    /api/admin/reports        - Get reported content
POST   /api/admin/posts/{id}/tags - Correct product tags
GET    /api/admin/analytics      - Platform analytics
```

9.2 Backend Tasks

- ☐ Create ModerationService
- ☐ Implement content flagging system
 - Duplicate detection (image hashing)
 - Text analysis for abuse
 - Product tag validation
- ☐ Build moderation queue with priority
- ☐ Create admin audit logging
- ☐ Implement bulk actions

9.3 Frontend - Admin Module (Web App)

- ☐ Create admin dashboard
 - ☐ Build moderation queue interface
 - Post preview
 - Approve/reject buttons
 - Feedback input
 - ☐ Create user management interface
 - ☐ Build platform analytics dashboard
 - ☐ Create content flagging review interface
-

Phase 10: Analytics & Event Tracking

10.1 Backend Tasks

- ☐ Create EventTrackingService
- ☐ Define event schema

```
- post.view
- post.like
- post.save
- post.share
- product.click
- product.convert
- feed.filter
- search.execute
- profile.complete
```

- ☐ Implement event batching and async processing
- ☐ Create analytics aggregation jobs (background service)
- ☐ Build real-time metrics endpoints
- ☐ Set up Application Insights / custom analytics

10.2 Frontend Tasks

- ☐ Create AnalyticsService wrapper
- ☐ Implement automatic page view tracking
- ☐ Add engagement event triggers
- ☐ Implement session tracking
- ☐ Add performance monitoring (Core Web Vitals)

Phase 11: Privacy & GDPR Compliance

11.1 Backend Tasks

- ☐ Implement data export endpoint (GDPR Article 15)
- ☐ Create account deletion with full data purge
- ☐ Build consent management system
- ☐ Implement data retention policies
- ☐ Create anonymization utilities
- ☐ Add audit trail for data access
- ☐ Implement cookie consent tracking

11.2 Frontend Tasks

- ☐ Create privacy settings page
- ☐ Build consent collection UI
- ☐ Implement data export request flow
- ☐ Create account deletion confirmation flow
- ☐ Add privacy policy acceptance tracking

Phase 12: Testing Strategy

12.1 Backend Testing

- ☐ Unit tests for services (xUnit)
 - MatchingService tests
 - FeedService tests
 - Analytics calculation tests
- ☐ Integration tests for API endpoints
- ☐ Database tests with test containers
- ☐ Load testing with k6 or similar
- ☐ Security testing (OWASP ZAP)

12.2 Frontend Testing

- ☐ Unit tests for services (Jasmine/Jest)
- ☐ Component tests (Angular Testing Library)
- ☐ E2E tests (Cypress or Playwright)
- ☐ Visual regression testing
- ☐ Performance testing (Lighthouse)

12.3 Test Coverage Targets

- Backend: 80% code coverage minimum
- Frontend: 70% code coverage minimum
- Critical paths: 100% coverage

Phase 13: Performance & Optimization

13.1 Backend Optimization

- ☐ Implement response caching
- ☐ Add Redis caching for:
 - Feed results
 - User profiles
 - Product data
- ☐ Optimize EF Core queries (no N+1)
- ☐ Implement database connection pooling
- ☐ Add pagination everywhere
- ☐ Implement request rate limiting

13.2 Frontend Optimization

- ☐ Implement lazy loading for modules
- ☐ Add image lazy loading
- ☐ Optimize bundle size (tree shaking)
- ☐ Implement virtual scrolling for feeds

- ☐ Add service worker for caching
 - ☐ Optimize images (WebP, srcset)
-

Phase 14: Deployment & DevOps

14.1 Infrastructure Setup

- ☐ Set up Azure/AWS infrastructure
 - App Service / ECS for API
 - Azure SQL / RDS for PostgreSQL
 - Blob Storage / S3 for media
 - Redis Cache
 - CDN for static assets
- ☐ Configure auto-scaling rules
- ☐ Set up monitoring and alerting
- ☐ Configure backup strategies

14.2 Mobile App Deployment

- ☐ Configure iOS build (Xcode, certificates)
 - ☐ Configure Android build (Gradle, signing)
 - ☐ Set up App Store Connect
 - ☐ Set up Google Play Console
 - ☐ Create app store assets (screenshots, descriptions)
 - ☐ Implement CodePush/OTA updates
-

Phase 15: MVP Launch Checklist

Pre-Launch

- ☐ Security audit completed
- ☐ Performance benchmarks met
- ☐ GDPR compliance verified
- ☐ App store compliance checked
- ☐ Legal review of terms/privacy policy
- ☐ Single retailer integration tested
- ☐ Creator cohort onboarded
- ☐ Content seeded for launch

Launch

- ☐ Soft launch to limited audience
- ☐ Monitor error rates and performance
- ☐ Gather initial user feedback
- ☐ Iterate on critical issues
- ☐ Full launch

Post-Launch

- ☐ Monitor KPIs (conversion, returns, engagement)
 - ☐ A/B test matching algorithm
 - ☐ Gather retailer feedback
 - ☐ Plan Phase 2 features
-

/## Appendix A: API Response Standards

```
// Success Response
{
  "success": true,
  "data": { },
  "meta": {
    "page": 1,
    "pageSize": 20,
    "totalCount": 100,
    "totalPages": 5
  }
}

// Error Response
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more validation errors occurred",
    "details": [
      { "field": "email", "message": "Email is required" }
    ]
  }
}
```

Appendix B: Folder Structure

Backend

```
/src
  /Lyke.Api
    /Controllers
    /Middleware
    /Filters
    Program.cs
  /Lyke.Core
    /Entities
    /Interfaces
    /Enums
    /Exceptions
```

```
/Lyke.Application
  /Services
  /DTOs
  /Validators
  /Mappings
/Lyke.Infrastructure
  /Data
    /Configurations
    /Migrations
    LykeDbContext.cs
  /Repositories
  /Services
    /Storage
    /Email
    /Analytics
/tests
  /Lyke.UnitTests
  /Lyke.IntegrationTests
```

Frontend (Ionic/Angular)

```
/src
  /app
    /core
      /services
      /guards
      /interceptors
    /shared
      /components
      /pipes
      /directives
    /features
      /auth
      /feed
      /profile
      /creator
      /settings
    /models
  /assets
  /environments
  /theme
```

Appendix C: Environment Variables

Backend

```
DATABASE_URL=postgresql://user:pass@host:5432/lyke
JWT_SECRET=<secret>
```



```
JWT_EXPIRY_MINUTES=60
REFRESH_TOKEN_EXPIRY_DAYS=30
AZURE_STORAGE_CONNECTION=<connection-string>
REDIS_CONNECTION=<connection-string>
GOOGLE_CLIENT_ID=<client-id>
APPLE_CLIENT_ID=<client-id>
```

Frontend

```
API_BASE_URL=https://api.lyke.app
GOOGLE_CLIENT_ID=<client-id>
APPLE_CLIENT_ID=<client-id>
ANALYTICS_KEY=<key>
```

Estimated Task Breakdown

Phase	Tasks	Completed	Priority	Status
Phase 1: Foundation	15	7	Critical	47% (backend only)
Phase 2: Database	8	7	Critical	88%
Phase 3: Authentication	17	12	Critical	71%
Phase 4: User Profile	18	13	Critical	72% (backend complete)
Phase 5: Content Feed	25	14	Critical	56% (backend complete)
Phase 6: Commerce	8	4	Critical	50% (backend complete)
Phase 7: Creator	14	0	High	0%
Phase 8: Retailer Portal	12	0	High	0%
Phase 9: Admin	10	0	High	0%
Phase 10: Analytics	8	0	Medium	0%
Phase 11: Privacy	8	0	Critical	0%
Phase 12: Testing	10	0	High	0%
Phase 13: Performance	8	0	Medium	0%
Phase 14: Deployment	10	0	High	0%
Phase 15: Launch	8	0	Critical	0%

Total: ~171 actionable tasks (~57 completed, ~33% overall)

Recommended MVP Sequence

1. **Foundation** (Phases 1-2): Project setup, database, basic infrastructure
2. **Core User Journey** (Phases 3-5): Auth, profiles, feed viewing
3. **Commerce** (Phase 6): Click tracking, attribution
4. **Content Creation** (Phase 7): Creator posting workflow
5. **Compliance & Launch** (Phases 11, 15): GDPR, testing, deployment

Retailer portal (Phase 8) and Admin tools (Phase 9) can run in parallel with a smaller team or be delivered incrementally.